

t during refinement, but we want to be transparent and accurate. If an issue is taking more effort than is reflected in the existing weight, the DRI on the issue is encouraged to change the weight. We want accurate documentation of the level of effort that was required.

Anything weighted as a 5 or larger should be broken down. These should not be ready for development.

Syntax deprecation process

Our CI syntax keeps evolving. We cannot support all keywords indefinitely, so deprecating and removing keywords is inevitable.

GitLab does not have a versioning system for CI/CD configuration. Therefore, it is critical to over-communicate our deprecation purposes to our users and take the necessary precautions to reduce the impact on their projects. Deprecating a keyword is risky because it will break all pipelines using it, and in some cases, users are not aware of the keyword they use in their pipeline. The deprecation process described below is similar to the deprecating and removing features process, with additional steps to reduce the risks which involved with removing a CI/CD keyword.

1. Deprecation notice - Syntax removal introduces a breaking change, as outlined in our deprecation process, we must notify the community and customers, which means including a deprecation notice in the monthly release post.

1. Track keyword usage - Tracking keyword usage should begin as early as possible. It is a mandatory step that helps estimate the user impact, timing, and needed effort. The more users use the keyword, the more time it takes to remove it (It took more than four years to move from remove to deprecation for 'type' keyword).

1. In-app warning - Provide our users with an in-app notification that we plan to remove a keyword they use in their pipeline. Our customers will get a notification in each run of the pipeline that uses the deprecated keyword. The warning will be printed:

- At run time on the pipeline page and logs.
- In the pipeline editor, while authoring a pipeline.

This step is optional if the keyword usage is relatively low (Recommend minimal reach of ~5% impacted users).

1. Keyword removal - The keyword will be removed from our code and schema and should happen in a major version. Once removed, using the keyword will result in a lint error.

Workflow

We use the Pipeline Authoring Workflow issue board to track what we work on in the current milestone.

We follow the product development flow to ensure that the problems we're solving are well understood and the solutions are well defined and validated before the implementation.

Our UX counterpart is the DRI for the SSOT designs, and they have the right to push any enhancement discussions in MRs into follow-up efforts in created and linked issues.

We use workflow labels to efficiently communicate an issue's state. Using these labels enables

collaboration across teams and communicates an issue's current state. Issues that have been refined but are not scheduled (are in Backlog) should have the workflow::scheduling label applied

The DRIs throughout each phase of the workflow are responsible for keeping the workflow labels up-to-date.

Async Issue Updates

In order to optimize async collaboration we use issue updates to share progress completed on a specific issue or epic.

Weekly updates on progress and status will be added to each issue by its assignee. A weekly update may be skipped if there was no progress. The update should be provided in the issue rather than in the related merge requests, as those do not provide a view of the overall progress. This applies to issues with the labels workflow::in dev, workflow::in review, or workflow::design.

The status comment can include what percentage complete the work is, the confidence of the person that their estimate is correct and, a brief note on what was done. It's perfectly acceptable to have multiple updates if more than one DRI is working on the issue.

As a part of the async update it's important to verify that the issue and related MRs workflow labels are correctly set.

Example

markdown

Async status update

- Complete: 80%
- Confidence: 90%
- Notes: expecting to go into review tomorrow

Aligning on feature development

The Engineering DRI works with the Product Designer throughout the workflow::in dev phase to uncover possible problems with the solution early enough that exhibit unexpected behaviour to what was originally agreed upon. If there are changes to be added that weren't agreed upon in the initial issue - a followup issue should be made and the Engineering DRI should work with the Product Manager to schedule that issue in a following iteration. This allows us to focus on cleanup over signoff, iterate quickly on issues with a low level of shame, and still make sure we accomplish what we've agreed upon. We should be careful not to hold off on completing these followup issues so that we don't build up a significant amount of Deferred UX issues.

If we find that solutions are consistently not matching the agreed upon design, we will hold a retrospective with the DRI, designer, and product manager to discuss where the gaps in communication are so that we can improve. It may be necessary to begin requiring a UX approval for merge requests on certain issues to help the Engineering DRI meet the requirements.

Technical Debt

- We track our team's technical debt using the following the Technical Debt Prioritization board, where we track issues with upcoming candidate labels and compare against current milestone.
- In an effort to promote more transparency across the stage where collaboration may be needed, starting in August 2024, we've introduced a Verify Pipeline teams Technical Debt board that shows the top of mind priorities in the Pipeline teams.

Retrospectives

{{% include "includes/engineering/verify-retrospectives.md" %}}

View the current Pipeline Authoring retrospective.

Team Communication

{{% engineering/verify-team-communication slack-channel="gpipeline-authoring" slack-url="https://gitlab.slack.com/archives/CPCJ8CCCX", grouplabel: 'group::pipeline authoring', groupissuesurl: 'https://gitlab.com/groups/gitlab-org/-/issues?labelname%5B%5D=group%3A%3Apipeline+authoring&scope=all', gitlabsubgroup: 'pipeline-authoring-group' %}}

Shared Team Calendar

The Pipeline Authoring team has a shared calendar that integrates with the gpipeline-authoring Slack channel to inform the team of important dates from Time Off by Deel.

Instructions on how to integrate with the Verify:Pipeline Authoring calendar can be found [here](#).

Developer Onboarding

Refer to the Developer Onboarding in Verify section.

SECTION: engineering/devops/ops/verify/pipeline-authoring/frontend-architecture.md

Overview

This page documents the frontend architecture process used in the Verify:Pipeline Authoring group.

Why Are We Doing This?

Other teams don't have an architecture process like this, you may be thinking. Why do we?

This process is the Pipeline Authoring group's approach for avoiding stumbling blocks we have encountered in the course of our work · late-breaking unexpected issues, lack of context, etc. This needs to be addressed by a process lighter than the Architecture Evolution Workflow and

more specific to the needs and knowledges of the Pipeline Authoring team than the frontend process.

By writing down what we plan to do, we hope to create a space where:

- Domain experts can point out unanticipated pitfalls early and share their knowledge.
- Context about goals and decisions are captured so that when people go on vacation / are eaten by a whale, all knowledge does not disappear with them.
- Engineers can do less switching between global thinking and detailed thinking when they are ready to implement their work.

When Do I Need to Use This Process?

This process should be used when an engineer is creating a new section; undertaking a significant refactor; or adding a large-ish feature that is not an extension of a existing pattern.

If you aren't sure if you need to do this, do it anyways! At worst you have spent some time thinking about your plan, and extra documentation never hurt anyone.

I'm Convinced! What Should This Look Like?

Add the label ~"Needs Architecture" to the feature issue, then link it to a new issue with a description containing:

- a description of the problem this work is intended to solve
- the goals for this work
- a proposal for the architecture, which may include one or many options

It does not need to address to the schedule. That can be handled in the traditional production issue.

The proposal should include:

- A component breakdown. This doesn't have to be perfect, but listing how, for instance, you might break up the work you are doing into sections, is good. This will feed into ...
- A data story! Where will the data we are displaying come from? Where does it go? How will subcomponents access it? What will the backend do, what will the frontend cover? Looking at the utility functions in the `app/assets/javascripts/pipelines/` and `app/assets/javascripts/pipelineeditor/` can help provide clues for what we already do.
- How these changes fit with the code that already exists. Does it follow the patterns? Does it diverge? Will it consume shared components?
- A list of unknowns. What don't you know? What are you unsure about? If this were a map, where would you write here be dragons?
- Finally, if any of the following base assumptions are to be contravened, this should be called out clearly and the reasoning explained.

This list may seem long, but for smaller changes, they should be able to be addressed in a sentence or two.

Base Assumptions For All Architectures

- We'll follow our other guidelines, both at the department level and those outlined in the team resources page.
- We will use Apollo and GraphQL for our data fetching and state handling needs.
- Using data structures that are identical to current data structures whenever possible is a good plan. While we do process a fair amount of data on the frontend, adding exceptions within that processing to deal with slight variations on that data is best avoided.
- We will use the reportFailure error pattern as used in, for instance, `app/assets/javascripts/pipelines/components/graph/graphcomponentwrapper.vue` or `app/assets/javascripts/pipelines/components/dag/dag.vue` to handle our errors.
- We will use a wrapper component at the top level, like `app/assets/javascripts/pipelines/components/graph/graphcomponentwrapper.vue`, to deal with the initial data calls and errors, and other functionality will be instantiated in subcomponents. This pattern helps us confine mocks in tests and supports the testing patterns without mounting too much.

Who Should Review?

Once you have your plan, share it with: the rest of the frontend team, backend, domain experts, product and UX.

No one on this list should be a blocker to the plan, but their input should be sought.

How Long Should This Take?

The process should last between one week and one milestone. If the project seems so large that it will take longer than a milestone, a spike might be needed.

SECTION: engineering/devops/ops/verify/pipeline-authoring/jtbd.md

Overview

The goal of this page is to document, share and iterate on the Jobs to be Done (JTBD) and their corresponding job statements for the Pipeline Authoring group. Using JTBD framework we intend to arrive at the more specific problems to be solved in relation to Pipeline authoring workflows.

Goals

Utilize JTBD and job statements to:

- Understand our users' underlying motivations
- Validate identified problem areas within the stage-group
- Create a common language across teams for better collaboration when working on improving the experience for GitLab CI
- Create a transparent view for our stakeholders into the current and future state of the product.

JTBD

{{% product/jtbd "Pipeline Authoring" %}}

SECTION: engineering/devops/ops/verify/pipeline-authoring/team-resources.md

Overview

The goal of this page is to document resources and recommendations for development patterns in the pipeline authoring group.

Frontend Architectural Planning

We use a lightweight architecture planning process for new features and significant refactors.

For more details see the architecture section.

CI camelCase Capitalization

When using camelCase we usually abbreviate "continuous integration" as Ci and not CI, this makes our file naming consistent, for example:

- MyCiComponent -> mycicomponent.vue
- myCiFileMutation -> mycifile.mutation.graphql

This matches the Ci in GraphQL types (e.g., CiJob, CiStatus) and other components.

Frontend Testing

We follow the Frontend development testing standards and style guidelines plus some recommendations to write tests.

Test fully rendered components

We use Vue Test Utils to test our Vue components implementation, and we prefer to test our components rendering their entire DOM structure, this allows us to:

- Ensure our tests actually test the result our users see
- Test integration with components from other libraries (like gitlab-ui)
- Use our tests to confirm implementation changes behave as expected instead of needing to update tests when the implementation changes
- Test as far as possible real events, so they are sensitive to event changes but not implementation changes
- See this example of our GitLab UI Combobox Spec

To support this while reaping the value of shallowMount where we can, we recommend the following approach:

1. Write tests using shallowMount while adhering to the "test rendered elements" guideline.
2. When shallowMount no longer renders the items that are to be tested, use mount
3. When tests start to show performance issues (such as regularly failing the default timeout), consider restructuring the tests to use partially rendered components or to test actions lower down on the mounting stack
 - As shallowMount this will stub most internal components by default, we will want to explicitly list some internal components as stubs, so the test will run with the real implementation of those components by adding them to the stubs option.

When switching from shallowMount to mount, we recommend making the method an argument to the createComponent function with shallowMount as the default to be sure mount is only being used when needed.

```
js
const createComponent = (options, mountMethod = shallowMount) => {
  wrapper = mountMethod(Component, {
    ...options
  })
}
```

This is a simple example of the approach:

```
js
// myformcontainerspec.js

import { GIButton, GIFORM } from '@gitlab/ui'
import { shallowMount } from '@vue/test-utils';
import MyFormContainer from '~/pipelineauthoring/myformcontainer.vue';

const createComponent = () => {
  wrapper = shallowMount(MyFormContainer, {
    // ...
    stubs: {
      GIFORM,
      GIButton, // the button will be fully rendered
    },
  });
};

// ...

const findCommitBtn = () => wrapper.find('[type="submit"]');
const findCommitBtnLoadingIcon = () => findCommitBtn().find(GILoadingIcon);

// ...

it('shows saving state', () => {
  expect(findCommitBtnLoadingIcon().exists()).toBe(true);
});
```

});

SECTION: engineering/devops/ops/verify/pipeline-execution/_index.md

This team maps to Verify DevOps stage and supports the Continuous Integration use case.

Vision

For an understanding of what this team is going to be working on take a look at the product vision. This team is responsible for delivering on the following directions:

- Continuous Integration
- Merge Trains

Mission

Make software development easier, faster, and more reliable by creating and supporting a Continuous Integration product that is performant, scalable, and lovable.

The Verify:Pipeline Execution Group is focused on supporting the functionality with respect to Continuous Integration. A key focus for the PE group is delivering features that achieve the outcome we track in our performance indicator.

Driving Factors

Performance

- Improving perceived performance from a customer perspective. For example, responsive UIs, reduced time to results.
- Reducing processing time in Pipeline Execution owned code.
- Ensuring the reliability of results. For example, return results to web requests in a timely manner (overlaps with scalability).

Scalability

- Supporting a lot of customers.
- Supporting a single customer with a large and/or complex set up.

Developer Efficiency

- Reducing the complexity of the codebase.
- Dealing with the breadth of the code owned by the team.
- Reducing length of time it takes to get an MR through the review process.
- Ensuring efficiency related to Community Contributions;
 - Ensuring that issues are approachable by the community.
 - Improving efficiency in the review process for community contributions.

Customer Experience

- Ensuring that customer issues are resolved in a timely manner.
- Addressing SUSImpacting issues in a timely manner.
- Ensuring reliable and accurate documentation is available.

Performance Indicator

We measure the value we contribute by using Performance Indicators (PI), which we define and use to track progress.

The current PI for the Pipeline Execution group is the number of unique users who trigger cpipelines. For more details, please check out the Product Team Performance Indicators. To view the latest Verify stage cipeline data see our Tableau Dashboard.

Usage Funnel

Based on the AARRR framework (Acquisition, Activation, Retention, Revenue, Referral), this funnel represents the customer journey in using GitLab CI. Each state in the funnel is defined with a metric to measure behavior. Product managers can focus on any of the various states in the funnel to prioritize features that drive a desired action.

mermaid

classDiagram

Acquisition --|> Activation

Acquisition: Awareness - Are users aware of CI pipelines?

Acquisition: Measurement (Total count of users in GitLab Projects with access to git write or "SCM Users")

Activation --|> Retention

Activation: Use - Are users creating CI configurations/pipelines?

Activation: Measurement (Total count of projects with .gitlab-ci.yml files)

Retention --|> Revenue

Retention: Continued use - Are users triggering CI pipelines?

Retention: Measurement (Total count of projects with a .gitlab-ci.yml file and CI pipeline triggered)

Revenue --|> Referral

Revenue: Monetary value - Are users paying for the CI features?

Revenue: Measurement (Total count of projects using paid CI features)

Referral --|> Acquisition

Referral: Endorsement - Are user encouraging others to use GitLab CI?

Referral: Measurement (Total count of Pipelines page views from social media))

Core domain

| Domain | Issues |

| ----- | ----- |

| Pipeline processing: processes responsible for transitions of pipelines, stages and jobs. |

~pipeline processing |

| Rails-Runner communication: jobs queuing, API endpoints and their underlying functionalities related to operations performed by and for Runners. | |

Continuous Integration domains

Domain	Issues
Continuous Integration and Deployment Admin Area settings	~CI/CD Settings
Repositories analytics for groups	~CI reports
GitLab CI/CD artifacts reports types	~CI reports
Unit test reports	~testing::code testing
Test with GitLab CI/CD and generate reports in merge requests	
Load Performance Testing	~testing::load performance
Metrics Reports	
Test coverage visualization	~testing::coverage
Browser Performance Testing	~testing::browser performance
Fail Fast Testing	
Accessibility testing	~testing::accessibility
Usability testing	~testing::usability
Review apps	~testing::review apps
Visual review tool (deprecated)	~testing::visual review tool
Scheduled pipelines	~pipeline schedules
Pipeline efficiency	~ci::scaling
Building images using Docker	
External SCM and CI Integration	
External pipeline validation	
Rate limits on pipeline creation	~Category:Continuous Integration + ~Eng-Inter-Dept::Rate Limits
Job logs	
Job log artifacts	
Merge Trains	~Category:Merge Trains

Not included in the Pipeline Execution group's domain:

- Secrets Management, see the direction page
- Pipeline Authoring, see the direction page
- Compliance in Pipelines, see the direction page
- Job artifacts: storage and management of artifacts is the gateway for many CI/CD features

Team Members

```
{{< team-by-manager-role role="Fullstack Engineering Manager, Verify:Pipeline Execution" >}}
```

Additional Team Responsibilities

These roles/responsibilities will be assigned to team members on a rotating basis to ensure we spread the load across the full team, and no individual has undue hardship by being the only "go-to" person for any of these tasks.

Front-line Responder

This team member will have the following responsibilities for the milestone (in priority order)

and will not have any assigned ~Deliverable items:

1. Respond to broken-master alerts for Pipeline Execution.
1. DRI for any incoming help requests from support.
 1. Existing help requests may be transferred to them at the discretion of the existing DRI.
 1. They are free to pull in others as needed to address the request, but remain the DRI for ensuring the requests are addressed.
1. First developer pulled into customer escalations unless there is an obvious reason to choose another team member.
1. [Optional] Organize and host a weekly swarm meeting to work on help requests with additional team members.
1. Respond to incoming questions in slack in pipeline-execution and s-verify (where related to Pipeline Execution).
 1. They can redirect to other team members as they deem necessary. The important part is to ensure we are responsive.
1. Investigate error budget overages.
1. Additional maintenance tasks (in suggested priority order):
 1. Address flaky tests as time permits.
 1. Work on long-standing maintenance issues as time permits.
 1. Example, we have a long list of very old feature flags that need to be addressed.
 1. Refinement of ~"type::maintenance" issues in ~"cicd::planning" as time permits.
 1. Any other ~"type::maintenance" issues in ~"cicd::active" as time permits.

For any of the above tasks, when an urgent issue arises outside of the team members working hours, other team members will need to be the first responders.

The milestone's Front-line Responder will be responsible for following up on such issues during their working hours.

They are not expected to be on-call 24/7 during the milestone.

At the end of the milestone, any issues that are still ongoing, may be transitioned to the incoming First Responder. If it makes sense for the original team member to continue being the DRI, this can also be done.

Community Coordinator

This team member will have the following responsibilities for the milestone in addition to their ~Deliverable task:

1. Do a first pass over any new community contributions as identified in the weekly Verify Pipeline teams: Community contributions report to ensure they seem like they are on the right track.

Rotation Schedule

Program Paused

Milestone	Front-line Responder	Community Coordinator

Stable Counterparts

To find our stable counterparts, look at the Pipeline Execution product category listing.

Technologies

Like most GitLab backend teams we spend a lot of time working in Rails on the main GitLab CE app, but we also do a lot of work in Go which is the language that GitLab Runner is written in. Familiarity with Docker and Kubernetes is also useful on our team.

Useful Links

- Issue tracker: ~group::pipeline execution
- Slack channel: gpipeline-execution
- GitLab unfiltered: Pipeline Execution group
- Grafana dashboard
- Tableau dashboard (to be migrated)
- Next planning issue
- Help Requests for pipeline execution
- Current milestone retro
- Needs weight board
- Current milestone board

Development Docs

For those new to the team, these links may be helpful in learning more about the product and technology.

- CI/CD Development Documentation

Engineering Overview

- CI Backend Architectural Walkthrough - May 2020
- Frontend CI product / codebase overview - June 2020

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="pipeline execution" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="pipeline execution" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="pipeline execution" >}}
```

```
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="pipeline execution" >}}  
{{< /tableau >}}
```

Cross-functional prioritisation

The team uses the `gpipelineexecutionquad` Slack channel to discuss cross-functional prioritisation in addition to any other topics that require the quad to collaborate on.

How We Work

The Pipeline Execution Workflow board is the source of truth for current and upcoming work.

Planning

Our planning timeline follows the GitLab Product Development timeline.

For information about Engineering Time see Engineering Initiatives.

PM (with help from EM as needed) will curate the list of Candidate::x.x + not Engineering Time list to ensure there is a reasonable number of high priority issues to select from at any point. If there are no issues for the milestone you are looking for you may select from a future milestone.

EM (with help from PM as needed) will maintain the list of Engineering Time + Candidate::x.x issues to ensure there is a reasonable number of high priority issues to select from at any point. If there are no issues for the milestone you are looking for you may select from a future milestone.

By the Friday 2 weeks before the end of the current milestone each Engineer will:

- Ensure the milestone on your current issue reflects the milestone you believe it will be completed based on the latest knowledge
 - Note: Any issues you have in progress that will not be completed in the current milestone can be used in lieu of selecting a new issue as long as the criteria matches
 - If you are unsure if the issues you have in progress are still a priority check with the EM and PM to verify.
 - Update the ~Deliverable or ~Stretch label if there is one if necessary
- Select 1 Candidate::x.x + not Engineering Time issue that you will work on in the upcoming milestone. Start with ~priority::1 issues first followed by ~priority::2 and ~priority::3
 - Assign yourself
 - Assure there is a weight set
 - Set the milestone according to the milestone when you expect the work to complete
 - If it is not the upcoming milestone, break the issue into multiple issues, one of which can be completed in the upcoming milestone if at all possible.
 - Let the EM/PM know you have done this in an issue comment on the original issue so they

can ensure the rest of the issues are planned for appropriately.

- Add the Deliverable label.
- Select 1 flaky-test issue to fix.
 - Do a quick check to see if there are multiple issues that are clearly caused by the same issue, link these as related and/or assign to themselves.
 - Set the milestone appropriately to indicate when the test will be fixed.
 - If you are unsure of the timeline, you can delay setting the milestone until you are comfortable committing to a milestone.
- Select 1 item to weight and refine for upcoming work
 - Assign this item to yourself. You can unassign it once you have completed the refinement process.
 - You may want to add a comment indicating that you are assigning to yourself for refinement purposes only, so there is no confusion. This is entirely optional.
 - Do not set a milestone on the issue.
 - If refinement is going to take considerable effort, create a refinement "~spike" issue (see also Spikes and Steps for Refining and Weighting Issues)
 - Tag the EM in a comment on the spike to let them know you have created this spike.
 - Use this as your Engineering Time item for the milestone.
- Select 1 item to weight and refine for community contribution
 - Assign this item to yourself. You can unassign it once you have completed the refinement process.
 - You may want to add a comment indicating that you are assigning to yourself for refinement purposes only, so there is no confusion. This is entirely optional.
 - Do not set a milestone on the issue.
 - If refinement is going to take considerable effort, create a refinement "~spike" issue (see also Spikes and Steps for Refining and Weighting Issues)
 - Tag the EM in a comment on the spike to let them know you have created this spike.
 - Use this as your Engineering Time item for the milestone.
- Select 1 Engineering Time + Candidate::x.x issue to fix.
- Assign yourself
 - Assure there is a weight set
 - Set the milestone according to the milestone when you expect the work to complete
 - If it is not the upcoming milestone, break the issue into multiple issues, one of which can be completed in the upcoming milestone if at all possible.
 - Let the EM/PM know you have done this in an issue comment on the original issue so they can ensure the rest of the issues are planned for appropriately.
 - add the Deliverable label or Stretch label.
- Add a comment to the milestone planning issue to indicate which issues that you have completed this planning work and the issues you have selected.

Throughout the milestone, if an Engineer completes their tasks and has extra capacity:

- checkout What do I work on next
- By the Monday the week the milestone ends:
 - Planning issue is finalized and PM will tag manager, SET/QEM, Product Design, engineers, and EM for review and feedback.
- By the Friday the milestone ends:
 - Engineers should comment on the planning issue with any changes to their planned work.

- PM completes monthly kick-off video featuring ~direction and Deliverable issues
- By the Monday of release week:
 - The EM will ensure that labels are set correctly for any milestone issues

Note: The EM and PM may need to modify the team commitments and schedule work for the upcoming milestone as we focus on Customer Results over what we plan.

How Engineering Refines Issues

side note: we prefer using Refining over Grooming

Engineers are expected to allocate approximately 6 hours each milestone to refine and weight issues on the ~needs weight board.

The purpose of refining an issue is to ensure the problem statement is clear enough to provide a rough effort sizing estimate; the intention is not to provide solution validation during refinement. When refining issues, engineers should timebox the activity to no more than 2 hours per issue. If an issue is complex and will require more research, we should track that effort in a refinement "~spike" to ensure we account for it in milestone planning. The "~spike" should be linked as blocking the original issue and outline the outcomes expected and a timebox for the effort should be specified. The original issue should be labeled as "~workflow::blocked".

Engineering uses the following handbook guidance for determining weights. If any issue needs any additional ~frontend ~backend ~Quality ~UX ~documentation reviews, they are assigned to the respective individual(s).

Checklist for Refining Issues

1. Does the issue have a problem statement in the description?
1. Does the issue have the expected behaviour described well enough for anyone to understand?
1. Does the issue explicitly define who the stakeholders are (e.g. BE, FE, PM, UX and/or Tech Writer)?
1. Does the issue have a proposal in the description? If so:
 1. Does the proposal address the problem statement?
 1. Are there any unintended side effects of the implementation?
1. Does the issue have proper labeling matching the job to be done? (e.g. bug, feature, performance)

Any one on the team can contribute to answering the questions in this checklist, but the final decisions are up to the PM and EMs.

Steps for Refining and Weighting Issues

Engineers will:

1. Go through the checklist above for refining issues assigned to them.
1. Split a backend issue out from the main issue if backend work needs to be completed before frontend work can begin.
1. Add a weight based on the definitions.

1. Update the ~workflow:: label to the appropriate status, e.g.
 - ~"workflow::design" if further design refinement is needed, and let the designer know.
 - ~"workflow::ready for development" when refinement is completed and a weight has been applied, signaling that it's ready for implementation and the issue can now be prioritized
 - ~"workflow::blocked" if more investigation and research is needed. Create a refinement "~spike" issue and ping the PM and EM.
1. Remove the "~needs weight" label if weighting was completed.
1. Unassign themselves from the issue when they are done refining and weighting the issue.

Timeboxing Issue Refinement

Because we should not be spending more than 2 hours per issue, we should re-evaluate if more time is needed or if a spike is required based on

1. If the issue is still reproducible on GitLab.com. If the cause is not obvious, a good first step is to ask the author for clarifications.
1. If Support engagement is present in the issue. We can tell if the issue is created by a GitLab team member or if it has the ~customer label.
1. If the issue is on GitLab.com or if it's on a self-managed instance. If this only affects self-managed instance, we can ask for request for help issue instead.
1. The scope and affect of the issue. If it's only affecting one customer or if it's present for multiple customers

A good general rule is to look at the P levels. P1 issues should be refined as they'll be immediately worked on in the next milestone. P2 and P3 can spin off spike issues if more time is needed.

Weighting Issues

The team makes use of the ~needs weight board that shows issues that need to be weighted for upcoming milestones. The criteria are denoted by having a ~needs weight and ~cid::planning applied. Throughout the month, team members review the ~needs weight board and assign issues to themselves. The priority order will be determined by column: Verify::P1, Verify::P2, Verify::P3, then 'Open'. If there is an issue with a higher urgency for weighting, a team member might be directly assigned to the issue for a prioritized review.

We add a Weight to issues as a way to estimate the effort needed to complete an issue. We factor in complexity and any additional coordination needed to work on an issue. We weight issues based on complexity, following the fibonacci sequence:

| Weight | Description |

| --- | --- | --- |

| 1: Trivial | The problem is very well understood, no extra investigation is required, the exact solution is already known and just needs to be implemented, no surprises are expected, and no coordination with other teams or people is required.

Examples are documentation updates, simple regressions, and other bugs that have already been investigated and discussed and can be fixed with a few lines of code, or technical debt that we know exactly how to address, but just haven't found time for yet. Around 1 MR to address |

| 2: Small | The problem is well understood and a solution is outlined, but a little bit of extra investigation will probably still be required to realize the solution. Few surprises are

expected, if any, and no coordination with other teams or people is required.

Examples are simple features, like a new API endpoint to expose existing data or functionality, or regular bugs or performance issues where some investigation has already taken place. Less than 2 MRs to address |

| 3: Medium | Features that are well understood and relatively straightforward. A solution will be outlined, and some edge cases will be considered, but additional investigation may be required to confirm the approach. Some surprises are expected, and coordination with other team members may be necessary.

Bugs that are relatively well understood, but additional investigation may be required. The expectation is that once the problem is verified and major edge cases have been identified, a solution should be relatively straightforward.

Examples are regular features, potentially with backend or frontend dependencies or performance issues. 3 or more MRs to address|

| 5: Large | Features that are well understood, but has more ambiguity and complexity. A solution will be outlined, and major edge cases will be considered, but additional investigation will likely be required to validate the approach. Surprises with specific edge cases are to be expected, and feedback from multiple engineers and/or teams may be required.

Bugs are complex and may be understood, and may not have an in depth solution defined during issue refinement. Additional investigation is likely necessary, and once the problem is identified, multiple iterations of a solution may be considered.

Examples are large features with backend and frontend dependencies, or performance issues that have a solution outlined but requires more in depth solution validation. Unknown amount of MRs to address, must be broken down.|

The maximum weighted value for an issue is a 5, and may exceed one milestone to complete given additional dependencies and/or complexity. Consider how an issue weighted with a 5 can be broken down into smaller iterations and do so.

A feature flag roll-out issue should be created while refining/weighting where anticipated. These are weight of 1.

Release Plans

To encourage more transparency and collaboration amongst the team and additionally align on the Release Posts we publish at the end of each milestone, we will be creating a separate issue to highlight a Feature flag roll out plan for each feature being released starting in 13.2, based on the issue template for feature flag roll outs. The engineer who implements the feature will be responsible for creating this separate issue to highlight the details of when and how the feature flag will be toggled, and subsequently link this issue to their feature issue. The product manager will tag this issue as a blocker to their release post, so that everyone is aligned on the release plan of the feature.

Prioritization of issues

The team makes use of Verify::P labels to indicate priority order of issues. As part of milestone planning and review we may decide to change or remove the labels to reflect new priority for issues. Currently these labels are applied as follows.

| Priority Label | Reason for applying |

| --- | --- |

| Verify::P1 | These are issues are the highest priority within their priority::x grouping. |

| Verify::P2 | These are issues are the second highest priority within their priority::x grouping. |
| Verify::P3 | These are issues are the third highest priority within their priority::x grouping. |

Workflow

We use the Pipeline Execution Workflow issue board to track what we work on in the current milestone.

Development moves through workflow states in the following order:

1. workflow::design (if applicable)
1. workflow::planning breakdown
1. workflow::ready for development
1. workflow::in dev
1. workflow::blocked (as necessary)
1. workflow::in review
1. workflow::verification
1. workflow::awaiting security release (if applicable)
1. workflow::feature-flagged (if applicable)
1. workflow::complete Closed

workflow::planning breakdown is driven by Product, but is a collaborative effort between Product and Engineering. The steps for planning breakdown typically consists of:

- Product defining or clarifying the problem statement. Product will collaborate on problem validation as needed.
- Engineering clarifying the issue description as stated. Then refining and weighting the issue. If there is not enough information to do so, the PM should be tagged on the issue and the workflow state should be set to workflow::problem validation.

At any point, if an issue becomes blocked, it would be in the workflow::blocked status. If there is a blocking issue, it needs to be added to the issue description or linked to the issue with a 'blocked by' relationship. If there is not a blocking issue, the reason for being blocked should be clearly communicated in a comment on the issue.

workflow::ready for development means that an issue has been sufficiently refined and weighted by Engineering. Issues in this state that are labeled cicd::active are ones that should be worked on in a milestone. When a developer starts working on an issue they should set the milestone to that of the one where the issue will most likely be completed, rather than the one where it is started.

workflow::awaiting security release is applied by an engineer after the security issue has passed verification and this label signals that it is ready for production but awaiting the next monthly security release. When this label is applied, the issue's milestone should also be updated to the next milestone to align with when the next security release will happen.

workflow::feature-flagged is applied to an issue that is being enabled through a separate feature flag rollout issue. Once the feature is validated the status is moved to

workflow::complete and the issue is closed.

Closed means that all code changes associated with the issue are fully enabled on gitlab.com. If it is being rolled out behind a feature flag, it means the feature flag is enabled for all users on gitlab.com.

More detail on the workflow is available on the Product-Development Flow page.

"What do I work on next?"

- check with your team members to see if they need help to complete their committed work. See Working Right to Left to reduce WIP
- check with the EM or PM to see if there are any customer support requests you can help with.
- consult the lists used in planning to find additional unclaimed items to work on.

What are the priorities for this milestone?

We use a series of labels to indicate the highest priority issues in the milestone.

1. The highest priorities for a given milestone would be issues labeled with Verify::P1, Deliverable, and group::pipeline execution to align with the Theme and Goals of the milestone.
1. Each milestone will start with 1 issue per engineer, which will be labeled as Deliverable, and group::pipeline execution. This should account for approximately 30% of the average total milestone weight.
1. Once all of the Verify::P1 issues have been picked up and are in workflow:in dev or beyond, we have Verify::P2 and Verify::P3 to signal issues that are important and will likely become Verify::P1 issues in later milestones.

Following the priorities, the lists used in planning will be curated, so that each team member can pull items from these lists as they choose according to capacity.

Setting the Milestone

When DRIs select issues, they will assign themselves to the issue and also add the milestone that they believe the issue will most likely be shipped, which may not be the current milestone. The DRI may work on issues which are in future milestones. If the milestone is set to a future milestone or not set, and you are sure it will be shipped in the current milestone, reset the milestone to the current one. This is also a good time to re-evaluate the weight and proposal, especially if the DRI picking up the issue was not the same individual who originally weighted and refined the issue. Aspirationally, we strive to iterate and want to break down the efforts to ship as much value in the milestone for our users as possible. This means if you see a more efficient way forward when you start working on a new issue, you are free to raise a comment and update the proposal to deliver more iterative value.

Working Right to Left to reduce WIP

Each member of the team can choose which issues to work on during a milestone by assigning the issue to themselves. When the milestone is well underway and we find ourselves looking for work, we default to working right to left on the issue board by pulling issues in the right-most column. If there is an issue that a team member can help with on the board, they should do

so instead of starting new work. This includes conducting code review on issues that the team member may not be assigned to, if they feel that they can add value and help move the issue along to completion. Additionally, prior to picking up the next issue from the top of the workflow::ready for development column, team members should check the ~needs weight board to ensure candidates for upcoming milestones have been weighted.

Specifically, this means our work is prioritized in the following order:

- Any verification on code that is in workflow::verification or workflow::production
- Conducting code reviews on issues that are workflow::in review
- Unblocking anyone in workflow::blocked or workflow::in dev if applicable
- Check the ~needs weight board for any issues needing to be weighted in current milestone.
- Then, picking from the top of the workflow::ready for development column

The goal of this process is to reduce the amount of work in progress (WIP) at any given time. Reducing WIP forces us to "Start less, finish more", and it also reduces cycle time. Engineers should keep in mind that the DRI for a merge request is the author(s), to reflect the importance of teamwork without diluting the notion that having a DRI is encouraged by our values.

Splitting issues

If an issue has several components (e.g. ~frontend, ~backend, or ~documentation) it may be useful to split it up into separate implementation issues.

The original issue should hold all the discussion around the feature, with the implementation issues being used to track the work done. If implementation will take more than 1 milestone and/or none of the implementation will be done against the original issue, it should be promoted to an epic.

Doing this provides several benefits:

1. There's only one DRI per issue.
1. Workflow labels and health statuses are more relevant.
1. We can weight issues with more accuracy.
1. We can mark one implementation as a blocker for another.
1. It's easier to see what work can be picked up by each functional group.
1. We can schedule feature work across multiple milestones.

When moving an issue through workflow::design to workflow::planning breakdown and implementation, use one of these processes:

Frontend?	Backend?	Action
-----	-----	-----
Yes	Yes	Original issue is renamed to a Frontend implementation issue and a separate Backend implementation issue is created.
No	Yes	Original issue is renamed to a Backend implementation issue.
Yes	No	Original issue is renamed to a Frontend implementation issue.

If an issue in workflow::design is too large in scope to be effectively implemented in one issue, or in case the issue is old and too cluttered with discussions, the issue can be broken down into smaller issues. The Product Designer will work together with the Engineers and the

Product Manager during workflow::design to understand the possible iteration plan and break down the large design proposal into smaller parts whenever possible.

When new implementation issues are created, they should always be linked to the initial issue that contains the proposal and relevant discussions.

Please use implementation issues responsibly.

They make sense for larger issues, but can be cumbersome for smaller features.

Weekly Issue Progress Updates

In order to keep our stakeholders informed of work in progress, we provide updates to issues either by updating the issue's health status and/or adding an async issue update.

Issue Health Status

For issues in the current milestone, we use the Issue Health Status feature to indicate probability that an issue will ship in the current milestone. This status is updated by the DRI (directly responsible individual) as soon as they recognize the probability has changed. If there is no change to the status, a comment to indicate that it has been the status of the issue has been assessed would be helpful.

The following are definitions of the health status options:

- On Track - The issue has no current blockers, and is likely to be completed in the current milestone.
- Needs Attention - The issue is still likely to be completed in the current milestone, but there are setbacks or time constraints that could cause the issue to miss the release due dates.
- At Risk - The issue is highly unlikely to be completed in the current milestone, and will probably miss the release due dates.

Examples of how status updates are added:

1. If the health status changes from On Track to Needs attention or At Risk, we recommend that the DRI add a short comment stating the reason for the change in an issue status update.
1. If an issue continues to be On Track, the DRI could provide a comment to indicate solutions (whatever it may be) continue to be implemented, and it's still on track to be delivered in the same milestone.

Issue status updates

When the DRI is actively working on an issue (workflow status is workflow::in dev, ~workflow::in review, or workflow::verification in the current milestone), they will add a comment into the issue with a status update, detailing:

- the updated issue health status
- notes on what was done based on the updated issue health status (especially if not On Track)
- anything else the DRI feels is beneficial to reflect the progress

There are several benefits to this approach:

- Team members can better identify what they can do to help the issue move along the board
- Creates an opening for other team members to engage and collaborate if they have ideas
- Leaving a status update is a good prompt to ask questions and start a discussion
- The wider GitLab community can more easily follow along with product development
- A history of the roadblocks the issue encountered is readily available in case of retrospection
- Product and Engineering Managers are more easily able to keep informed of the progress of work

Expectations for DRIs when providing updates for work in progress:

- Status updates are provided once per week, barring special circumstances (e.g. PTO)
- Ideally updates are made at a logical part of a DRI's workflow to minimize disruption, and not necessarily at the same time/day each week
 - Generally when there is a logical time to leave an update, such as a change in issue health status, that is the best time to do so
 - Can be used to present some technical findings or information relevant to other stakeholders

Tracking Inactive Issues

As a general rule, any issues being actively worked on have one of the following workflow labels:

- workflow::in dev
- workflow::in review
- workflow::verification
- workflow::complete (upon closing the issue)

The Health Status of these issues should be updated to:

1. Needs Attention - on the 1st of the month.
1. At Risk - on the 8th of the month.

EMs are responsible for manually updating the Health Status of any inactive issues in the milestone accordingly.

Spikes

Spikes are time-boxed investigations typically performed in agile software development. We create Spike issues when there is uncertainty on how to proceed on a feature from a technical perspective before a feature is developed.

Guidelines

- Spikes are typically marked as deliverable to ensure clear ownership from engineers
- Spikes are typically time-boxed to a short duration, sometimes one week, never more than one milestone.

- We should try to schedule Spikes at the beginning of an iteration so that the investigation will conclude before the planning of the following iteration.
- Limit no more than 2 Spikes per milestone per group.
- Typically multiple team members collaborate on a spike. We want to ensure we get multiple different viewpoints as well as keep the research focused and efficient.
- Spikes should be assigned to at least one frontend and one backend engineer to ensure we consider the issue from both viewpoints.

Category Labels

The Pipeline Execution group supports the product marketing categories described below:

Label				
-----	-----	----	-----	---
Category:Continuous Integration	Issues	MRs	Direction	Documentation
Category:Merge Trains	Issues	MRs	Direction	Documentation

Feature Labels

Label			Description
-----	-----	----	-----
api	Issues	MRs	Issues related to API endpoints for CI features.
CI permissions	Issues	MRs	Issues related to CIJOBTOKEN and CI authentication
Compute minutes	Issues	MRs	All issues and MRs related to how we count continuous integration minutes and calculate usage. Formerly ~ci minutes
merge requests	Issues	MRs	Issues related to CI functionality within the Merge Request.
notifications	Issues	MRs	Issues related to various forms of notifications related to CI features.
pipeline analytics	Issues	MRs	Issues related to CI pipeline statistics and dashboards.
pipeline processing	Issues	MRs	Issues related to the execution of pipeline jobs, including DAG, child pipelines, and matrix build jobs

Other Notable Labels

Label			Description
-----	-----	----	-----
CI/CD core platform	Issues	MRs	Any issues and merge requests related to CI/CD core domain, either as changes to be made or as observable side effects.
onboarding	Issues		Issues that are helpful for someone onboarding as a new team member.
Good for 1st time contributors	Issues		Issues that are good for first time community contributors, and could similarly be labeled for onboarding
Seeking community contributions	Issues		Issues we would like the wider community to contribute and may also be labeled Seeking community contributions

Developing with Feature Flags

When building features that may have high impact the team uses established GitLab guidelines

for feature flags.

We also ensure we are collaborating with our teammates in customer support and customer success by alerting them to the rollout issue before a feature is enabled.

The feature flags introduced by the team still in the code can be found in this table.

You can also search all Feature Flags through Sam's great tool here (prefiltered for pipeline execution)

Collaboration with UX and Engineering

To create a high-quality product that is functional and useful · Engineering, PM and Product Designer work closely together, combine methodologies, and often connect throughout the product development process. They use workflow::design to discuss possible complexities, challenges, and uncover blockers around the proposed solution.

Product Designers play a critical role in the product development of user-facing issues. They collaborate with the Engineering and the Product Manager to design the user experience for the features. Once the design solution is proposed, agreed and validated, the Engineering DRI is assigned to implement that design and functionality during the milestone for which the issue is planned.

Following the code review guidelines means that we should ensure all MRs with user facing changes are reviewed by a product designer. UX reviews should follow the designer review guidelines as closely as possible to reduce the impact on velocity while maintaining quality.

Inclusive Development

Our process of planning and development relies heavily on overcommunication rather than any approval gates or automated notification mechanisms. We adhere to the proactive mindset and responsibility of everyone involved to make sure every step in the process is as transparent as it can be. For both planning and building this means direct, cross-functional, and other relevant stakeholders are included early into the process. This ensures everyone is able to contribute to the best of their capabilities, and at the right time in the process. This can include, but is not limited to, GitLab objects, Slack, meetings, and daily stand-ups.

Some practical examples of this are:

- When you open up or begin working on an epic, issue, or merge request, consider if all stakeholders are aware of this or should be updated. If unsure, error on the side of updating rather than keeping silent.
- When making significant progress make sure this is seen by the relevant stakeholders even if you don't require immediate feedback by mentioning them rather than relying on automated email notifications. Make their involvement explicit.

Note: For issues related to Merge Request experience, always keep the Code Review group in the loop to avoid any technical or Deferred UX from occurring. Refer to the collaboration on merge requests experience page to learn more about the collaboration framework.

Note: A good practice when only wanting to inform rather than requesting a direct action from the mentioned stakeholders is to put FYI directly following the @mention handle.

UX Scorecards

A top priority for us is usability, and one way to effectively evaluate our JTBDs is with periodic UX Scorecards. For technical tasks that require infrastructure support, such as a functional cluster or provisioned environment with a .gitlab-ci.yml file, the Product Designer and the Product Manager can work with the Engineering Manager and Quality stable counterparts to craft a project based on the scenarios to test for the JTBDs. Some guidelines for working together in this case:

- Be sure to schedule at least a full milestone in advance, so there is enough lead time for the stable counterparts
- Create an overall epic and add issues for task tracking for infrastructure creation
- Schedule a kick off or record a walkthrough of the scenarios for everyone to have a shared understanding of the JTBD that are being evaluated

We are also working on maintaining a library of precreated tasks for contribution from the engineering team in the CI Sample Projects group. These will help avoid having too much overhead from UX Scorecards and Category Maturity Scorecards in the future.

Breaking down issues for iteration

We follow the steps below to achieve the best results in the shortest time:

1. After user research or customer feedback identifies a user need, and has generated some ideas to meet that need, the Product Designer drives the process of moving those ideas into design proposals involving the PM and the Engineering Team early and often. The UX Definition of Done can be referenced to give counterparts better insight into which steps in the design workflow need to be completed before the MVC can move to the development phase.
 1. Depending on our confidence in the proposed solution, we may take those prototypes through user testing to validate if they would solve the original problem.
 1. Engineers, the Product Designer, EM and PM can compare and contrast multiple possible engineering approaches to decide which one to work on first based on the scope of the solutions and experience of the team.
 1. When a solution is validated and the team has a clear understanding of the effort required, it is time to break the big solution down into smaller issues. This is usually done in the workflow::planning breakdown phase and the whole team (PM, Engineers, Product Designer, QA, and Technical Writer) is involved in the process. They work closely together to find the most technically feasible and smallest feature set to deliver value to early customers and provide feedback for future product development. Check out iteration strategies for help.

We aim to design broadly for an epic or full feature at least one milestone ahead of time and then break the big solution into smaller issues to pick up in the next milestones. Suppose working one milestone ahead to design the big solution is not possible. In that case, Engineering and Product Designer will define the first most technically feasible and smallest feature set (MVC) to satisfy early customers that will be implemented in the same milestone.

Avoiding crunch times between UX, Product and Engineering

- Ideally, Product Management and Product Designers aim to work 3 months in advance of Engineering proposals to ensure the problem definition and solution has been adequately validated prior to building. See Validation track for more details. This allows us to come up with the bigger idea ahead of time, and work further with Engineering to break it down into smaller iterations. Ideally, this should be completed before the implementation milestone starts.
- the Product Designer, PM, and Engineering use the Design phase in the Validation track to talk about complexities and discuss challenges and uncover blockers. Once we are all in agreement, we can move it to the Solution Validation phase.
- If it is taking more than a week to understand and investigate the technical feasibility for the design solution, update the workflow label to ~workflow::blocked and change the assignee to engineering DRIs until the technical discussion is resolved. If the discussion is expected to go on longer, reducing the chances of the design solution being delivered in the intended milestone, consider creating a spike issue for the discussion that blocks the current issue.
- Engineers and Product Designers should stay in contact and frequently align throughout the Build track to avoid unplanned changes.

Collaborating with the Wider Community on Community Merge Requests

For more details on how to contribute to GitLab generally, please see our documentation.

Aligning on feature development

The Engineering DRI works with the Product Designer throughout the workflow:in dev phase to uncover possible problems with the solution early enough that exhibit unexpected behaviour to what was originally agreed upon. If there are changes to be added that weren't agreed upon in the initial issue - a followup issue should be made and the Engineering DRI should work with the Product Manager to schedule that issue in a following milestone. This allows us to focus on cleanup over signoff, iterate quickly on issues with a low level of shame, and still make sure we accomplish what we've agreed upon. We should be careful not to hold off on completing these followup issues so that we don't build up a significant amount of Deferred UX issues.

If we find that solutions are consistently not matching the agreed upon design, we will hold a retrospective with the DRI, designer, and product manager to discuss where the gaps in communication are so that we can improve. It may be necessary to begin requiring a UX approval for merge requests on certain issues to help the Engineering DRI meet the requirements.

Collaborating with other Product Groups at GitLab

We strive to be as collaborative as possible in our work because the scope of our product area touches many other groups, including Create:Source Code and Verify:Runner. To enable collaboration we may work together with other internal stakeholders on a single MR to update or create a feature and documentation. <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/76253> is an example of how this can work very well.

E2E Tests

Whenever possible, engineers should add all relevant tests alongside a new feature or a bug fix.

It is always required that engineers should add all relevant tests (unit, component,

integration or E2E) alongside a new feature or a bug fix.

We recognize, however, that in the case of creating E2E tests this cannot be applied systematically for multiple reasons. This section lists some ways we work to deliver tests in a painless and efficient way.

Plan ahead

We aim to define needed tests early with Quad-planning.

All the testing should be defined before the implementation starts and all parties should agree on:

- What type of test coverage is needed.
- Who is the DRI for which test.
- What is to be delivered.
- When it will be delivered.
- If Feature flag is required consider strategy for testing feature flags in end to end tests

E2E tests are written in parallel

When writing a new feature, we might need to write new E2E specs. In Pipeline Execution, we prefer

to add E2E tests in separate MRs, the same way we prefer frontend and backend MRs to be separate.

During Quad-planning, it is essential

to determine whether that separate MR is required for the feature to ship or not. Given that we use feature flags for all

new features, it is quite easy to work in separate MRs and turn on the flag when the team feels the feature has enough coverage

for production use. A typical full stack feature can therefore involve multiple backend MRs, then frontend MRs, and finally

E2E test MRs.

Best person for the job

Writing tests is a team effort and should never be delegated to a single group. W